

AI ASSISTED CODING

ASSIGNMENT-11.1

HTNO: 2303A510B2

NAME: K AKHIL

BATCH: 14

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

PROMPT:

“#generate a Stack class with push, pop, peek, and is_empty methods. Use a list internally. Add proper docstrings and comments.”

CODE:

```
#generate a Stack class with push, pop, peek, and is_empty methods. Use a list internally. Add proper docstrings and comments.
class Stack:
    """
    A Stack is a data structure that follows the Last In First Out (LIFO) principle.
    This implementation uses a list to store the elements of the stack.
    """

    def __init__(self):
        """Initialize an empty stack."""
        self.stack = []

    def push(self, item):
        """
        Add an item to the top of the stack.

        Parameters:
        item: The item to be added to the stack.
        """
        self.stack.append(item)

    def pop(self):
        """
        Remove and return the item at the top of the stack.

        Returns:
        The item at the top of the stack.

        Raises:
        IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Pop from an empty stack")
        return self.stack.pop()

    def peek(self):
        """
        Return the item at the top of the stack without removing it.

        Returns:
        The item at the top of the stack.

        Raises:
        IndexError: If the stack is empty.
        """
        if self.is_empty():
```

```

    if self.is_empty():
        raise IndexError("Peek from an empty stack")
    return self.stack[-1]

def is_empty(self):
    """
    Check if the stack is empty.

    Returns:
    True if the stack is empty, False otherwise.
    """
    return len(self.stack) == 0
# Example usage in user input with a simple command loop to interact with the stack.
if __name__ == "__main__":
    stack = Stack()
    while True:
        command = input("Enter command (push, pop, peek, is_empty, quit): ").strip().lower()
        if command == "push":
            item = input("Enter item to push: ")
            stack.push(item)
            print(f"'{item}' pushed to stack.")
        elif command == "pop":
            try:
                item = stack.pop()
                print(f"'{item}' popped from stack.")
            except IndexError as e:
                print(e)
        elif command == "peek":
            try:
                item = stack.peek()
                print(f"Top item: '{item}'")
            except IndexError as e:
                print(e)
        elif command == "is_empty":
            print("Stack is empty." if stack.is_empty() else "Stack is not empty.")
        elif command == "quit":
            print("Exiting the program.")
            break
        else:
            print("Invalid command. Please try again.")

```

OUTPUT:

```

PS C:\Users\akhil\OneDrive\Documents\AI(vs)> & C:/Users/akhil/AppData/Local/Programs/P
Enter command (push, pop, peek, is_empty, quit): pop
Pop from an empty stack
Enter command (push, pop, peek, is_empty, quit): peek
Peek from an empty stack
Enter command (push, pop, peek, is_empty, quit): is_empty
Stack is empty.
Enter command (push, pop, peek, is_empty, quit): quit
Exiting the program.
PS C:\Users\akhil\OneDrive\Documents\AI(vs)>

```

JUSIFICATION:

Stack follows the LIFO principle, where the last inserted element is removed first. It is useful in undo operations, recursion, and expression evaluation.

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

PROMPT:

“#Implement a Queue using Python lists. Create a FIFO Queue class in Python using list with enqueue, dequeue, peek and size methods. provide the code in user input”

CODE:

```
#Implement a Queue using Python lists.Create a FIFO Queue class in Python using list with enqueue, dequeue, peek and size methods.provide the code in user input
class Queue:
    """
    A Queue is a data structure that follows the First In First Out (FIFO) principle.
    This implementation uses a list to store the elements of the queue.
    """

    def __init__(self):
        """Initialize an empty queue."""
        self.queue = []

    def is_empty(self):
        """
        Check if the queue is empty.

        Returns:
            True if the queue is empty, False otherwise.
        """
        return len(self.queue) == 0

    def enqueue(self, item):
        """
        Add an item to the end of the queue.

        Parameters:
            item: The item to be added to the queue.
        """
        self.queue.append(item)

    def dequeue(self):
        """
        Remove and return the item at the front of the queue.

        Returns:
            The item at the front of the queue.

        Raises:
            IndexError: If the queue is empty.
        """
        if self.is_empty():
            raise IndexError("Dequeue from an empty queue")
        return self.queue.pop(0)
```

```

def peek(self):
    """
    Return the item at the front of the queue without removing it.

    Returns:
    The item at the front of the queue.

    Raises:
    IndexError: If the queue is empty.
    """
    if self.is_empty():
        raise IndexError("Peek from an empty queue")
    return self.queue[0]

def size(self):
    """
    Return the number of items in the queue.

    Returns:
    The number of items in the queue.
    """
    return len(self.queue)

```

```

# Example usage in user input. Give numbers to select.
if __name__ == "__main__":
    queue = Queue()
    while True:
        print("\nSelect an operation:")
        print("1. Enqueue")
        print("2. Dequeue")
        print("3. Peek")
        print("4. Size")
        print("5. Exit")

        choice = input("Enter your choice: ")

        if choice == '1':
            item = input("Enter item to enqueue: ")
            queue.enqueue(item)
            print(f"Enqueued: {item}")
        elif choice == '2':
            try:
                item = queue.dequeue()
                print(f"Dequeued: {item}")
            except IndexError as e:
                print(e)
        elif choice == '3':
            try:
                item = queue.peek()
                print(f"Front of the queue: {item}")
            except IndexError as e:
                print(e)
        elif choice == '4':
            size = queue.size()
            print(f"Size of the queue: {size}")
        elif choice == '5':
            print("Exiting...")
            break
        else:
            print("Invalid choice. Please try again.")

```

OUTPUT:

```
1. Enqueue
2. Dequeue
3. Peek
4. Size
5. Exit
Enter your choice: 2
Dequeued: AKHIL

Select an operation:
1. Enqueue
2. Dequeue
3. Peek
4. Size
5. Exit
Enter your choice: 2
Dequeued: PRAVEEN

Select an operation:
1. Enqueue
2. Dequeue
3. Peek
4. Size
5. Exit
Enter your choice: 2
Dequeue from an empty queue

Select an operation:
1. Enqueue
2. Dequeue
3. Peek
4. Size
5. Exit
Enter your choice: 5
Exiting...
```

JUSTIFICATION:

Queue follows the FIFO principle, where the first inserted element is removed first. It is commonly used in scheduling and real-time processing systems.

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

PROMPT:

“#Generate a singly linked list in Python with Node class and LinkedList class.

#Include insert and display methods with comments.”

CODE:

```
#Generate a singly linked list in Python with Node class and LinkedList class.
#Include insert and display methods with comments.
class Node:
    """
    A Node is a basic building block of a linked list.
    It contains data and a reference to the next node in the list.
    """

    def __init__(self, data):
        """Initialize a node with data and set the next reference to None."""
        self.data = data
        self.next = None

class LinkedList:
    """
    A LinkedList is a data structure that consists of a sequence of nodes.
    Each node points to the next node in the list, allowing for dynamic memory allocation.
    """

    def __init__(self):
        """Initialize an empty linked list with head set to None."""
        self.head = None

    def insert(self, data):
        """
        Insert a new node with the given data at the end of the linked list.

        Parameters:
        data: The data to be stored in the new node.
        """
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def display(self):
        """
        Display the contents of the linked list.

        This method traverses the linked list and prints the data of each node.
        """
        current_node = self.head
        while current_node:
            print(current_node.data, end=' ')
            current_node = current_node.next
        print() # for a new line after displaying the list
```

```

print() # For a new line after displaying the list
# Example usage in user input.
if __name__ == "__main__":
    linked_list = LinkedList()
    while True:
        user_input = input("Enter a number to insert into the linked list: ")
        if user_input.lower() == 'exit':
            break
        try:
            number = int(user_input)
            linked_list.insert(number)
            print("Current Linked List:")
            linked_list.display()
        except ValueError:
            print("Please enter a valid integer.")

```

OUTPUT:

```

PS C:\Users\akhil\OneDrive\Documents\AI(vs)> & C:/Users/akhil/AppData/Local/Programs/Python/Python39-64/Scripts/python.exe C:/Users/akhil/OneDrive\Documents\AI\src\linked_list.py
Enter a number to insert into the linked list: 10
Current Linked List:
10
Enter a number to insert into the linked list: 20
Current Linked List:
10 20
Enter a number to insert into the linked list: 30
Current Linked List:
10 20 30

```

JUSTIFICATION:

Linked List allows dynamic memory allocation and easy insertion or deletion. It is useful when the size of data changes frequently.

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

PROMPT:

“#Create a Binary Search Tree class in Python with recursive insert and inorder traversal methods.

#Include documentation.”

CODE:

```
#Create a Binary Search Tree class in Python with recursive insert and inorder traversal methods.
#Include documentation.
class TreeNode:
    """
    A TreeNode is a node in a binary search tree.
    It contains data and references to the left and right child nodes.
    """

    def __init__(self, data):
        """Initialize a tree node with data and set left and right references to None."""
        self.data = data
        self.left = None
        self.right = None

class BinarySearchTree:
    """
    A BinarySearchTree is a data structure that maintains a sorted order of elements.
    Each node has at most two children, and the left child is less than the parent node,
    while the right child is greater than the parent node.
    """

    def __init__(self):
        """Initialize an empty binary search tree with root set to None."""
        self.root = None

    def insert(self, data):
        """
        Insert a new node with the given data into the binary search tree.

        Parameters:
        data: The data to be stored in the new node.
        """
        if self.root is None:
            self.root = TreeNode(data)
        else:
            self._insert_recursive(self.root, data)

    def _insert_recursive(self, node, data):
        """Helper method to insert a new node recursively."""
        if data < node.data:
            if node.left is None:
                node.left = TreeNode(data)
            else:
                self._insert_recursive(node.left, data)
        else:
            if node.right is None:
                node.right = TreeNode(data)
            else:
                self._insert_recursive(node.right, data)

    def inorder_traversal(self):
        """
        Perform an inorder traversal of the binary search tree.

        Returns:
        A list of elements in sorted order.
        """
        return self._inorder_recursive(self.root)

    def _inorder_recursive(self, node):
        """Helper method to perform inorder traversal recursively."""
        elements = []
        if node:
            elements.extend(self._inorder_recursive(node.left))
            elements.append(node.data)
            elements.extend(self._inorder_recursive(node.right))
        return elements
```

```
# Example usage in user input.
if __name__ == "__main__":
    bst = BinarySearchTree()
    while True:
        user_input = input("Enter a number to insert into the binary search tree (or 'exit' to quit): ")
        if user_input.lower() == 'exit':
            break
        try:
            number = int(user_input)
            bst.insert(number)
            print("Inorder Traversal of the Binary Search Tree:")
            print(bst.inorder_traversal())
        except ValueError:
            print("Please enter a valid integer.")
```


OUTPUT:

```
PS C:\Users\akhil\OneDrive\Documents\AI(vs)> & C:/Users/akhil/AppData/Local/Programs/Python/Python11.1/Python.exe C:/Users/akhil/AppData/Local/Programs/Python/Python11.1/Scripts/python C:/Users/akhil/AppData/Local/Programs/Python/Python11.1/Scripts/python (vs)/lab(11.1)/t4.py"
Enter a number to insert into the binary search tree (or 'exit' to quit): 50
Inorder Traversal of the Binary Search Tree:
[50]
Enter a number to insert into the binary search tree (or 'exit' to quit): 30
Inorder Traversal of the Binary Search Tree:
[30, 50]
Enter a number to insert into the binary search tree (or 'exit' to quit): 70
Inorder Traversal of the Binary Search Tree:
[30, 50, 70]
Enter a number to insert into the binary search tree (or 'exit' to quit): 20
Inorder Traversal of the Binary Search Tree:
[20, 30, 50, 70]
Enter a number to insert into the binary search tree (or 'exit' to quit): 40
Inorder Traversal of the Binary Search Tree:
[20, 30, 40, 50, 70]
Enter a number to insert into the binary search tree (or 'exit' to quit): abc
Please enter a valid integer.
```

JUSTIFICATION:

BST maintains elements in sorted order for efficient searching. It provides faster insert and search operations compared to linear structures.

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

PROMPT:

“#Implement a hash table class with basic insert, search, and delete methods.
#also include Collision handling using chaining, with well-commented methods.”

CODE:

```

# Implement a hash table class with basic insert, search, and delete methods.
# Also include Collision handling using chaining, with well-commented methods.
class HashTable:
    """
    A HashTable is a data structure that stores key-value pairs.
    This implementation uses chaining to handle collisions, where each bucket contains a list of key-value pairs.
    """

    def __init__(self, size=10):
        """Initialize the hash table with a specified size and create empty buckets."""
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Generate a hash value for the given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """
        Insert a key-value pair into the hash table.

        Parameters:
        key: The key to be inserted.
        value: The value associated with the key.
        """
        index = self._hash(key)
        bucket = self.table[index]

        # Check if the key already exists in the bucket
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value) # Update existing key
                return

        # If the key does not exist, add a new key-value pair
        bucket.append((key, value))

    def search(self, key):
        """
        Search for a value associated with the given key.

        Parameters:
        key: The key to be searched.
        """

```

```

Returns:
The value associated with the key if found, None otherwise.
"""
index = self._hash(key)
bucket = self.table[index]

for k, v in bucket:
    if k == key:
        return v # Return the value if the key is found

return None # Return None if the key is not found

def delete(self, key):
    """
    Delete a key-value pair from the hash table.

    Parameters:
    key: The key to be deleted.

    Returns:
    True if the deletion was successful, False if the key was not found.
    """
    index = self._hash(key)
    bucket = self.table[index]

    for i, (k, v) in enumerate(bucket):
        if k == key:
            del bucket[i] # Remove the key-value pair from the bucket
            return True

    return False # Return False if the key was not found

```

```

# Example usage in user input
if __name__ == "__main__":
    hash_table = HashTable()
    while True:
        command = input("Enter command (insert, search, delete, quit): ").strip().lower()
        if command == "insert":
            key = input("Enter key to insert: ")
            value = input("Enter value to insert: ")
            hash_table.insert(key, value)
            print(f"Inserted ({key}, {value}) into the hash table.")
        elif command == "search":
            key = input("Enter key to search: ")
            value = hash_table.search(key)
            if value is not None:
                print(f"Value found for key '{key}': {value}")
            else:
                print(f"Key '{key}' not found in the hash table.")
        elif command == "delete":
            key = input("Enter key to delete: ")
            if hash_table.delete(key):
                print(f"Key '{key}' deleted from the hash table.")
            else:
                print(f"Key '{key}' not found in the hash table.")
        elif command == "quit":
            break
        else:
            print("Invalid command. Please enter 'insert', 'search', 'delete', or 'quit'.")

```

OUTPUT:

```
PS C:\Users\akhil\OneDrive\Documents\AI(vs)> & C:/Users/akhil/AppData/Local
● Enter command (insert, search, delete, quit): insert
Enter key to insert: 101
Enter value to insert: akhil
Inserted (101, akhil) into the hash table.
Enter command (insert, search, delete, quit): search
Enter key to search: 101
Value found for key '101': akhil
Enter command (insert, search, delete, quit): search
Enter key to search: 102
Key '102' not found in the hash table.
Enter command (insert, search, delete, quit): delete
Enter key to delete: 101
Key '101' deleted from the hash table.
Enter command (insert, search, delete, quit): quit
```

JUSTIFICATION:

Hash Table provides very fast lookup, insertion, and deletion using keys. It is ideal for applications requiring quick data retrieval.

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

PROMPT:

“Create a Graph class using adjacency list with methods add_vertex, add_edge, and display.”

CODE:

```

lab(11.1) > t6.py > ...
1  #Create a Graph class using adjacency list with methods add_vertex, add_edge, and display.
2  class Graph:
3      """
4      A Graph is a data structure that consists of a set of vertices and edges.
5      This implementation uses an adjacency list to represent the graph.
6      """
7
8      def __init__(self):
9          """Initialize an empty graph with an adjacency list."""
10         self.adjacency_list = {}
11
12     def add_vertex(self, vertex):
13         """
14         Add a vertex to the graph.
15
16         Parameters:
17         vertex: The vertex to be added to the graph.
18         """
19         if vertex not in self.adjacency_list:
20             self.adjacency_list[vertex] = []
21
22     def add_edge(self, vertex1, vertex2):
23         """
24         Add an edge between two vertices in the graph.
25
26         Parameters:
27         vertex1: The first vertex.
28         vertex2: The second vertex.
29         """
30         if vertex1 in self.adjacency_list and vertex2 in self.adjacency_list:
31             self.adjacency_list[vertex1].append(vertex2)
32             self.adjacency_list[vertex2].append(vertex1) # For undirected graph
33
34     def display(self):
35         """Display the adjacency list of the graph."""
36         for vertex, edges in self.adjacency_list.items():
37             print(f"{vertex}: {edges}")

```

```

# Example usage in user input. give numbering
if __name__ == "__main__":
    graph = Graph()
    while True:
        print("1. Add Vertex")
        print("2. Add Edge")
        print("3. Display Graph")
        print("4. Exit")
        choice = input("Enter your choice: ")
        if choice == '1':
            vertex = input("Enter vertex to add: ")
            graph.add_vertex(vertex)
        elif choice == '2':
            vertex1 = input("Enter the first vertex: ")
            vertex2 = input("Enter the second vertex: ")
            graph.add_edge(vertex1, vertex2)
        elif choice == '3':
            graph.display()
        elif choice == '4':
            break
        else:
            print("Invalid choice. Please try again.")

```

OUTPUT:

```
PS C:\Users\akhil\OneDrive\Documents\AI(vs)> & C:/Users/akhil/AppD
(vs)/lab(11.1)/t6.py"
4. Exit
Enter your choice: 1
Enter vertex to add: B
1. Add Vertex
2. Add Edge
3. Display Graph
4. Exit
Enter your choice: 2
Enter the first vertex: A
Enter the second vertex: B
1. Add Vertex
2. Add Edge
3. Display Graph
4. Exit
Enter your choice: 3
A: ['B']
B: ['A']
```

JUSTIFICATION:

Graph represents relationships between nodes using edges. It is suitable for networks, routes, and connected systems.

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

PROMPT:

"Implement a priority queue in Python using heapq module with enqueue, dequeue and display methods."

CODE:

1) > t7.py > ...

```
#Implement a priority queue in Python using heapq module with enqueue, dequeue and display methods.
import heapq
class PriorityQueue:
    """
    A PriorityQueue is a data structure that allows for efficient retrieval of the highest priority item.
    This implementation uses the heapq module to maintain a min-heap, where the smallest item has the highest priority.
    """

    def __init__(self):
        """Initialize an empty priority queue."""
        self.elements = []

    def is_empty(self):
        """
        Check if the priority queue is empty.

        Returns:
        True if the priority queue is empty, False otherwise.
        """
        return not self.elements

    def enqueue(self, item, priority):
        """
        Add an item to the priority queue with a given priority.

        Parameters:
        item: The item to be added to the priority queue.
        priority: The priority of the item (lower values indicate higher priority).
        """
        heapq.heappush(self.elements, (priority, item))

    def dequeue(self):
        """
        Remove and return the item with the highest priority from the priority queue.

        Returns:
        The item with the highest priority.

        Raises:
        IndexError: If the priority queue is empty.
        """
        if self.is_empty():
            raise IndexError("Dequeue from an empty priority queue")
        return heapq.heappop(self.elements)[1]
```

```

def display(self):
    """Display the contents of the priority queue."""
    print("Priority Queue Contents:")
    for priority, item in sorted(self.elements):
        print(f"Priority: {priority}, Item: {item}")
# Example usage in user input . Give numbering to easy access.
if __name__ == "__main__":
    pq = PriorityQueue()
    while True:
        print("1. Enqueue")
        print("2. Dequeue")
        print("3. Display")
        print("4. Exit")
        choice = input("Enter your choice: ")
        if choice == '1':
            item = input("Enter item to enqueue: ")
            priority = int(input("Enter priority: "))
            pq.enqueue(item, priority)
        elif choice == '2':
            try:
                item = pq.dequeue()
                print(f"Dequeued item: {item}")
            except IndexError as e:
                print(e)
        elif choice == '3':
            pq.display()
        elif choice == '4':
            break
        else:
            print("Invalid choice. Please try again.")

```

OUTPUT:


```
PS C:\Users\akhil\OneDrive\Documents\AI(vs)> & C:/Users/akhil/AppData/Local/Programs/Python/Python
(vs)/lab(11.1)/t7.py"
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter item to enqueue: akhil
Enter priority: 2
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter item to enqueue: praveen
Enter priority: 1
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter item to enqueue: bharath
Enter priority: 3
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Priority Queue Contents:
Priority: 1, Item: praveen
Priority: 2, Item: akhil
Priority: 3, Item: bharath
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Dequeued item: praveen
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Dequeued item: akhil
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
```

JUSTIFICATION:

Priority Queue processes elements based on priority instead of order of arrival. It is used in scheduling and shortest path algorithms.

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using `collections.deque`.

PROMPT:

“Generate a Deque class in Python using collections.deque. Include methods to insert at front, insert at rear, remove from front, remove from rear, display elements, and add proper docstrings.”

CODE:

```
#Generate a Deque class in Python using collections.deque.Include methods to insert at front, insert at rear,
#remove from front, remove from rear, display elements,and add proper docstrings.
from collections import deque
class Deque:
    """
    A Deque (Double-Ended Queue) is a data structure that allows insertion and removal of elements from both ends.
    This implementation uses the collections.deque class for efficient operations.
    """

    def __init__(self):
        """Initialize an empty deque."""
        self.deque = deque()

    def insert_front(self, item):
        """
        Insert an item at the front of the deque.

        Parameters:
        item: The item to be inserted at the front.
        """
        self.deque.appendleft(item)

    def insert_rear(self, item):
        """
        Insert an item at the rear of the deque.

        Parameters:
        item: The item to be inserted at the rear.
        """
        self.deque.append(item)

    def remove_front(self):
        """
        Remove and return the item at the front of the deque.

        Returns:
        The item at the front of the deque.

        Raises:
        IndexError: If the deque is empty.
        """
        if not self.deque:
            raise IndexError("Remove from an empty deque")
        return self.deque.popleft()
```

```

def remove_rear(self):
    """
    Remove and return the item at the rear of the deque.

    Returns:
    The item at the rear of the deque.

    Raises:
    IndexError: If the deque is empty.
    """
    if not self.deque:
        raise IndexError("Remove from an empty deque")
    return self.deque.pop()

def display(self):
    """Display the elements in the deque."""
    print("Deque contents:", list(self.deque))

# Example usage in user input
if __name__ == "__main__":
    dq = Deque()
    while True:
        print("1. Insert at Front")
        print("2. Insert at Rear")
        print("3. Remove from Front")
        print("4. Remove from Rear")
        print("5. Display")
        print("6. Exit")
        choice = input("Enter your choice: ")
        if choice == '1':
            item = input("Enter item to insert at front: ")
            dq.insert_front(item)
        elif choice == '2':
            item = input("Enter item to insert at rear: ")
            dq.insert_rear(item)
        elif choice == '3':
            try:
                removed_item = dq.remove_front()
                print(f"Removed from front: {removed_item}")
            except IndexError as e:
                print(e)
        elif choice == '4':
            try:
                removed_item = dq.remove_rear()
                print(f"Removed from rear: {removed_item}")
            except IndexError as e:
                print(e)
        elif choice == '5':
            dq.display()
        elif choice == '6':
            break
        else:
            print("Invalid choice, please try again.")

```

OUTPUT:

```

(vs)/lab(11.1)/c0.py
1. Insert at Front
2. Insert at Rear
3. Remove from Front
4. Remove from Rear
5. Display
6. Exit
Enter your choice: 1
Enter item to insert at front: 10
1. Insert at Front
2. Insert at Rear
3. Remove from Front
4. Remove from Rear
5. Display
6. Exit
Enter your choice: 2
Enter item to insert at rear: 20
1. Insert at Front
2. Insert at Rear
3. Remove from Front
4. Remove from Rear
5. Display
6. Exit
Enter your choice: 5
Deque contents: ['10', '20']
1. Insert at Front
2. Insert at Rear
3. Remove from Front
4. Remove from Rear
5. Display
6. Exit
Enter your choice: 3
Removed from front: 10
1. Insert at Front
2. Insert at Rear
3. Remove from Front
4. Remove from Rear
5. Display
6. Exit
Enter your choice: 6
PS C:\Users\akhil\OneDrive\Documents\AI(vs)>

```

JUSTIFICATION:

Deque allows insertion and deletion from both ends. It is flexible and can work as both stack and queue.

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

PROMPT:

“Generate a Python program for Cafeteria Order Management using Queue data structure with enqueue, dequeue, display methods, and menu-driven user input. Add proper docstrings and comments.”

CODE:

```
lab(11.1) > 9.py > ...
1  #Generate a Python program for Cafeteria Order Management using Queue data structure with enqueue, dequeue, display methods, and menu-driven user input.
2  #Add proper docstrings and comments.
3  class CafeteriaOrderManagement:
4      """
5      A Cafeteria Order Management system that uses a Queue data structure to manage customer orders.
6      This implementation allows for enqueueing new orders, dequeueing completed orders, and displaying the current orders in the queue.
7      """
8
9      def __init__(self):
10         """Initialize an empty order queue."""
11         self.orders = []
12
13     def enqueue(self, order):
14         """
15         Add a new order to the end of the queue.
16
17         Parameters:
18         order: The order to be added to the queue (e.g., a string representing the customer's order).
19         """
20         self.orders.append(order)
21
22     def dequeue(self):
23         """
24         Remove and return the order at the front of the queue.
25
26         Returns:
27         The order at the front of the queue.
28
29         Raises:
30         IndexError: If the order queue is empty.
31         """
32         if not self.orders:
33             raise IndexError("Dequeue from an empty order queue")
34         return self.orders.pop(0)
35
36     def display(self):
37         """Display the current orders in the queue."""
38         if not self.orders:
39             print("No current orders in the queue.")
40         return
```

```

41         print("Current Orders in Queue:")
42         for index, order in enumerate(self.orders, start=1):
43             print(f"{index}. {order}")
44     # Example usage in user input
45     if __name__ == "__main__":
46         order_management = CafeteriaOrderManagement()
47         while True:
48             print("\nCafeteria Order Management System")
49             print("1. Enqueue Order")
50             print("2. Dequeue Order")
51             print("3. Display Orders")
52             print("4. Exit")
53             choice = input("Enter your choice: ")
54
55             if choice == '1':
56                 order = input("Enter the customer's order: ")
57                 order_management.enqueue(order)
58                 print("Order added to the queue.")
59
60             elif choice == '2':
61                 try:
62                     completed_order = order_management.dequeue()
63                     print(f"Completed Order: {completed_order}")
64                 except IndexError as e:
65                     print(e)
66
67             elif choice == '3':
68                 order_management.display()
69
70             elif choice == '4':
71                 print("Exiting the system. Goodbye!")
72                 break
73
74             else:
75                 print("Invalid choice. Please try again.")

```

OUTPUT:

```
PS C:\Users\akhil\OneDrive\Documents\AI(vs)> & C:/Users/akhil/AppData/Local/Programs/Python/Python313/python.exe "c:\
Cafeteria Order Management System
1. Enqueue Order
2. Dequeue Order
3. Display Orders
4. Exit
Enter your choice: 1
Enter the customer's order: akhil
Order added to the queue.

Cafeteria Order Management System
1. Enqueue Order
2. Dequeue Order
3. Display Orders
4. Exit
Enter your choice: 1
Enter the customer's order: jashwanth
Order added to the queue.

Cafeteria Order Management System
1. Enqueue Order
2. Dequeue Order
3. Display Orders
4. Exit
Enter your choice: 3
Current Orders in Queue:
1. akhil
2. jashwanth

Cafeteria Order Management System
1. Enqueue Order
2. Dequeue Order
3. Display Orders
4. Exit
Enter your choice: 2
Completed Order: akhil

Cafeteria Order Management System
1. Enqueue Order
2. Dequeue Order
3. Display Orders
4. Exit
Enter your choice: 3
Current Orders in Queue:
1. jashwanth

Cafeteria Order Management System
1. Enqueue Order
2. Dequeue Order
3. Display Orders
4. Exit
Enter your choice: 4
Exiting the system. Goodbye!
PS C:\Users\akhil\OneDrive\Documents\AI(vs)> |
```

JUSTIFICATION:

1 Student Records – Dictionary

Stores data using unique ID for fast access.

2 Course Enrollment – List

Allows dynamic addition and removal of courses.

3 Faculty Information – Dictionary

Provides quick lookup using Faculty ID.

Campus Events – List

Stores multiple events and supports easy updates.

Task Description #10: Smart E-Commerce Platform – Data Structure

Challenge

PROMPT:

“Generate a Python program for Top-Selling Products Tracker using Priority Queue (heapq module). Include methods to add product sales, get top-selling product, display ranking, and menu-driven interface. Add proper docstrings and comments.”

CODE:

```
lab(11.1) > t10.py > ...
1  """Generate a Python program for Top-Selling Products Tracker using Priority Queue (heapq module).
2  # Include methods to add product sales, get top-selling product, display ranking, and menu-driven interface.
3  # Add proper docstrings and comments."""
4  import heapq
5  class TopSellingProductsTracker:
6      """
7      A Top-Selling Products Tracker that uses a Priority Queue to manage product sales.
8      This implementation allows for adding product sales, retrieving the top-selling product, and displaying the ranking of products.
9      """
10
11     def __init__(self):
12         """Initialize an empty priority queue for tracking product sales."""
13         self.product_sales = []
14
15     def add_product_sale(self, product_name, sales):
16         """
17         Add a product sale to the tracker.
18
19         Parameters:
20         product_name: The name of the product.
21         sales: The number of sales for the product.
22         """
23         heapq.heappush(self.product_sales, (-sales, product_name)) # Use negative sales for max-heap behavior
24
25     def get_top_selling_product(self):
26         """
27         Retrieve the top-selling product.
28
29         Returns:
30         The name of the top-selling product.
31
32         Raises:
33         IndexError: If there are no products in the tracker.
34         """
35         if not self.product_sales:
36             raise IndexError("No products in the tracker")
37         return self.product_sales[0][1] # Return the product name with the highest sales
38
39     def display_ranking(self):
40         """Display the ranking of products based on sales."""
41         if not self.product_sales:
42             print("No products in the tracker.")
43             return
44         print("Top-Selling Products Ranking:")
45         for rank, (sales, product_name) in enumerate(sorted(self.product_sales), start=1):
46             print(f"{rank}. {product_name} - Sales: {-sales}")
```



```

47 # Example usage in user input
48 if __name__ == "__main__":
49     tracker = TopSellingProductsTracker()
50     while True:
51         print("\nTop-Selling Products Tracker")
52         print("1. Add Product Sale")
53         print("2. Get Top-Selling Product")
54         print("3. Display Ranking")
55         print("4. Exit")
56         choice = input("Enter your choice: ")
57
58         if choice == '1':
59             product_name = input("Enter the product name: ")
60             sales = int(input("Enter the number of sales: "))
61             tracker.add_product_sale(product_name, sales)
62             print("Product sale added.")
63
64         elif choice == '2':
65             try:
66                 top_product = tracker.get_top_selling_product()
67                 print(f"The top-selling product is: {top_product}")
68             except IndexError as e:
69                 print(e)
70
71         elif choice == '3':
72             tracker.display_ranking()
73
74         elif choice == '4':
75             print("Exiting the program.")
76             break
77
78         else:
79             print("Invalid choice. Please try again.")

```

OUTPUT:

```

Top-Selling Products Tracker
1. Add Product Sale
2. Get Top-Selling Product
3. Display Ranking
4. Exit
Enter your choice: 1
Enter the product name: laptop
Enter the number of sales: 50
Product sale added.

Top-Selling Products Tracker
1. Add Product Sale
2. Get Top-Selling Product
3. Display Ranking
4. Exit
Enter your choice: 1
Enter the product name: mobile
Enter the number of sales: 80
Product sale added.

Top-Selling Products Tracker
1. Add Product Sale
2. Get Top-Selling Product
3. Display Ranking
4. Exit
Enter your choice: 2
The top-selling product is: mobile

Top-Selling Products Tracker
3. Display Ranking
4. Exit
Enter your choice: 2
The top-selling product is: mobile

...

Top-Selling Products Tracker
4. Exit
Enter your choice: 2
The top-selling product is: mobile

Top-Selling Products Tracker
Enter your choice: 2
The top-selling product is: mobile

Top-Selling Products Tracker
The top-selling product is: mobile

Top-Selling Products Tracker

Top-Selling Products Tracker
1. Add Product Sale
2. Get Top-Selling Product
3. Display Ranking
4. Exit
Enter your choice: 3
Top-Selling Products Ranking:
1. mobile - Sales: 80
2. laptop - Sales: 50

```

JUSTIFICATION:

1 Shopping Cart – Linked List

Allows dynamic add/remove of products easily.

2 Order Processing – Queue

Processes orders in FIFO order.

3 Top-Selling Products – Priority Queue

Retrieves highest selling products first.

4 Product Search Engine – Hash Table

Provides fast lookup using product ID.

5 Delivery Route Planning – Graph

Represents warehouses and locations as connected nodes.
